

# AI Blog Writer + Scheduled Blogs

## Setup & Operations Guide

A complete, readable version of the supplied setup document. All substantive source content is retained.

# AI Blog Writer + Scheduled Blogs — setup

Two halves talking through Supabase:

- **The admin screens** (/admin/blog-writer, /admin/blog-schedules) let an editor queue topics and press "Write next post". That only records a request row — the browser cannot write anything.
- **The watcher** (npm run blog:watch) is a long-running Node process on a machine that stays online and stays logged into a **Claude subscription** (never a metered API key). It polls Supabase for requests and due schedules, spawns the Claude CLI under a restricted tool allowlist, and the spawned session researches, writes, audits and imports the post as a draft (or published, if auto-publish is on and the audit found zero errors).

**Only one watcher may ever point at one Supabase project.** The DB-level single-flight index and `claim_next_write_request()` stop two of them from double-writing the same post, but two watchers is still not a supported configuration — restart the service, don't run a second instance.

**If this VPS will ever run more than one blog-writer-style watcher** (this app plus another app's), give each a distinct systemd unit name and log path from day one. A generic `blog-writer.service` name has already caused a real collision on a shared box once. This doc uses `ghlprime-blog-writer` throughout specifically to avoid that.

## 1. Run the schema (once)

Supabase dashboard → SQL Editor → New query → paste  
`supabase/blog-writer-schema.sql` → Run. It is idempotent (safe to re-run) and creates:

- `blog_topics` (the queue), `blog_write_requests` (the run log with phase/steps/usage), `blog_run_schedules`, `blog_deleted_schedules` (soft-delete snapshots), `blog_writer_settings` (key/value)
- 7 optional columns on `blog_posts` (`target_keyword`, `cta_variant`, `sources`, `research_mode`, `source_url`, `source_name`, `topic_id`)
- the `claim_next_write_request()` function (atomic single-flight claim)
- the public `blog-images` storage bucket + its policies (stock photos are copied there so the site never hotlinks a third party)

Every writer table is to authenticated only — an anonymous visitor sees

nothing. The admin screens and the watcher both operate as a signed-in Supabase Auth user.

If the SQL editor refuses the storage policies on your project ("must be owner of table objects"), create the bucket by hand: Storage → New bucket → blog-images, Public → and add four policies on it: SELECT for public, INSERT / UPDATE / DELETE for authenticated. Note that `supabase.storage.getBucket()` reports "Bucket not found" to a normal authenticated user even when the bucket exists (it needs read access to `storage.buckets`); test with an upload instead:

```
node -e "import('./scripts/blog-writer/supabaseClient.js').then(async (m) => { const sb = await m.getAuthenticatedClient(); c
```

## 2. Prerequisites on the VPS

- Node.js 20+ and npm
- A clone of this repo (`git clone ... && cd ghlprime && npm install`)
- Outbound internet access (research, Supabase, the `claude` CLI)

## 3. Install and log in to Claude Code

```
npm install -g @anthropic-ai/claude-code # or the native installer
claude auth login # interactive: prints a URL + code
# on a headless box:
claude setup-token
claude auth status # confirm it took
```

**Smoke-test unattended execution before anything else**, from inside the repo:

```
claude -p "Reply with exactly: OK"
```

You should get OK and exit code 0. **If this fails, stop here** — nothing below will work until it passes.

The watcher finds the binary on PATH, then `~/local/bin`, then the VS Code extension's bundled copy. Set `CLAUDE_BIN=/path/to/claude` to override.

## 4. Environment variables

Create `.env` in the repo root (git-ignored already):

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key

# A dedicated Supabase Auth login for the writer — NOT a person's admin
# login. Create it in Supabase Auth > Users. It needs no role beyond
# "authenticated" (the same role every admin login already has).
BLOG_WRITER_EMAIL=writer@ghlprime.com
BLOG_WRITER_PASSWORD=...
```

```
# Stock photos for blog:import (one of these). Without either, posts simply
# get no images rather than failing.
PEXELS_API_KEY=...
# UNSPLASH_ACCESS_KEY=...

# Optional: lets blog:import trigger a sitemap refresh after auto-publishing.
# Matches the existing api/refresh-sitemap.js token pattern.
SITE_MAP_REFRESH_TOKEN=...
SITE_URL=https://ghlprime.com

# Optional: the model the writing session uses (default: sonnet). The
# headline ranker for "sources" schedules always uses haiku.
BLOG_WRITER_MODEL=sonnet
```

SEED\_EMAIL / SEED\_PASSWORD (used by the older scripts/seed-\*.mjs) are accepted as fallbacks for the writer login.

## 5. systemd service

/etc/systemd/system/ghlprime-blog-writer.service:

```
[Unit]
Description=GHL Prime AI blog writer watcher
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=YOUR_APP_USER
WorkingDirectory=/path/to/ghlprime
ExecStart=/usr/bin/npm run blog:watch
Restart=always
RestartSec=15
# Claude Code reads its login from this user's home directory.
Environment=HOME=/home/YOUR_APP_USER
StandardOutput=append:/var/log/ghlprime-blog-writer.log
StandardError=append:/var/log/ghlprime-blog-writer.log

[Install]
WantedBy=multi-user.target
```

Run it as the same user that logged Claude Code in — the credentials live in that user's home directory, and running as anyone else fails authentication.

```
sudo touch /var/log/ghlprime-blog-writer.log && sudo chown YOUR_APP_USER /var/log/ghlprime-blog-writer.log
sudo systemctl daemon-reload
sudo systemctl enable --now ghlprime-blog-writer
sudo systemctl status ghlprime-blog-writer
tail -f /var/log/ghlprime-blog-writer.log
```

Restart the service (`systemctl restart`) rather than stop-then-start. A restart is handled like any other interruption: on its next start the watcher parks anything it left running as waiting and picks it straight back up. There is no special stop procedure.

## 6. Verify end to end

1. Open `/admin/blog-writer` — the badge should say **Online** within about

30 seconds (the heartbeat is every 15s; "offline" means no beat for 2 minutes).

2. Add a topic and press **Write next post**.

3. The card switches to "Writing now" and the phase list advances as the writer actually calls tools (reading the standard → researching → writing → auditing → images → saving). Those phases come from the session's real tool calls, not a timer.

4. `tail -f /var/log/ghlprime-blog-writer.log` — expect picked up request within 30 seconds (the poll interval), then the phases, then request ... done (slug) and a cost line.

5. `/admin/blog` — the post is at the top as a draft (unless auto-publish is on). The draft JSON is kept in `content/drafts/imported/`.

## What happens when things go wrong

| Situation                                              | What the watcher does                                                                                                                                                                                                                                         |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude usage limit hit                                 | Parks the request as <b>**waiting**</b> with a <code>`retry_after`</code> (the stated reset time when the message carries one, otherwise 15 → 30 → 60 min backoff), then resumes by itself. Shown as "Paused", never as a failure. Up to 6 automatic retries. |
| Run exceeds 25 minutes                                 | Same: waiting + retry.                                                                                                                                                                                                                                        |
| Topic has nothing worth writing / the run breaks on it | Request marked <b>**failed**</b> , the topic marked <b>**skipped**</b> with the reason, and the queue moves on to the next one. Retry from the screen puts it back.                                                                                           |
| Watcher restarted mid-run                              | The abandoned <code>`running`</code> row is parked as waiting and picked up on the next poll.                                                                                                                                                                 |
| A batch ("Write all" or a schedule's day)              | One request at a time, chained; Stop clears the switch and the chain ends after the current post. A schedule's unwritten keywords go back to the front of its list.                                                                                           |
| Schedule due while the machine was asleep              | Fires if it is less than 6 hours late; otherwise writes the day off and waits for tomorrow. Never fires twice in a day.                                                                                                                                       |

## Where things live

| Piece                                        | Path                                                                                                                                    |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Schema                                       | <code>`supabase/blog-writer-schema.sql`</code>                                                                                          |
| Pure rule modules (shared by browser + Node) | <code>`scripts/blog-writer/*.js`</code>                                                                                                 |
| Writing standard / prompts                   | <code>`scripts/blog-writer/prompts.js`</code> (printed by <code>`npm run blog:rules`</code> )                                           |
| CLI scripts the writer session runs          | <code>`scripts/blog-queue.mjs`</code> , <code>`blog-rules.mjs`</code> , <code>`blog-audit.mjs`</code> , <code>`blog-import.mjs`</code>  |
| Watcher                                      | <code>`scripts/blog-watch.mjs`</code>                                                                                                   |
| Workflow the spawned session follows         | <code>`.claude/commands/write-blog.md`</code>                                                                                           |
| Admin screens                                | <code>`src/pages/AdminBlogWriterPage.jsx`</code> , <code>`AdminBlogSchedulePage.jsx`</code> , <code>`src/components/blogWriter/`</code> |

| Piece                                                                                          | Path                                                                                            |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Admin data layer                                                                               | <code>`src/lib/blogWriterApi.js`</code>                                                         |
| Public CTA banner + sources fold                                                               | <code>`src/components/BlogCtaBanner.jsx`</code> ,<br><code>`src/lib/blogContentSplit.js`</code> |
| Sheet-import proxy (CORS workaround; served in dev by <code>`vite-plugins/dev-api.js`</code> ) | <code>`api/import-sheet.js`</code>                                                              |
| Unit smoke tests                                                                               | <code>`npm run blog:test`</code>                                                                |

## Known gap: prerendered HTML / sitemap freshness

This site prerenders each `/blog/:slug` to static HTML at **build time** (Puppeteer, in `npm run build`) for SEO, and snapshots published posts into `src/data/contentSnapshot.json`. A post the watcher publishes is live immediately for real visitors (the SPA reads Supabase directly), but has no prerendered static HTML or sitemap entry until the next Vercel deploy. `SITEMAP_REFRESH_TOKEN` only refreshes `sitemap.xml`. Triggering a full redeploy (a Vercel Deploy Hook) after auto-publishing is a deliberate follow-up, not yet wired in.